Lecture 6 — Sweeps & Segment Trees in Practice

> Regional Level · Prerequisites: Year Curriculum Weeks 7 (prefix sums, difference arrays), 23–25 (segment trees, Fenwick, lazy propagation) · Estimated time: 2.5 hours

1. Overview

You know how segment trees and Fenwick trees work. At regionals the challenge is seeing that a problem about rectangles, segments, intervals, or offline queries becomes easy once you **sweep** and keep the right structure for the cross-section. This lecture covers:

- coordinate compression, including compressing *intervals* when lengths matter,
- the sweep-line recipe and, crucially, the order of events at equal coordinates,
- offline query processing by sorting queries,
- four segment-tree patterns that recur in practice: cover counting, descent, mergeable nodes, and value-indexed trees.

2. Coordinate compression

When coordinates are large (up to 10^9) but there are only n of them, replace each by its rank:

```
vector<int> xs = all_values;
sort(xs.begin(), xs.end());
xs.erase(unique(xs.begin(), xs.end()), xs.end());
auto id = [&](int v) { return int(lower_bound(xs.begin(), xs.end(), v) - xs.begin()); };
```

Compression preserves order and discards distances. If you need lengths or areas, compress endpoints and let index `i` represent the **elementary interval** `[xs[i], xs[i+1]]`; its true length is `xs[i+1] − xs[i]`. A segment `[a, b]` then covers elementary intervals `id(a) … id(b) − 1`.

Compress every coordinate that will be looked up, including query coordinates. If a query value might not appear in the list, use `upper_bound(...) − 1` to find "the largest compressed value ≤ v".

3. The sweep-line recipe

1. Choose a sweep direction (x, or time).
2. Convert each object into **events** at the coordinates where the cross-section changes: an interval becomes an opening and a closing event; a vertical segment becomes a single query event.
3. Sort the events. At equal coordinates, the order must follow the problem's semantics.
4. Maintain the cross-section in a data structure (Fenwick tree, segment tree, multiset) and answer queries as the line passes.

3.1 Event order at equal coordinates

This is where most sweep solutions fail. Decide explicitly:

Situation	Order at equal coordinate
Closed intervals: touching intervals overlap	open before close
Half-open intervals `[a, b)`: touching ones do not overlap	close before open
Segment endpoints touching a query segment count	open → query → close
A query "at time t" sees updates at time t	update → query

Encode the order as a small integer in the event tuple, e.g. `(x, type, ...)` with types 0, 1, 2, and plain `sort` enforces it.

CSES 1619 Restaurant Customers is the 1D warm-up: events `(arrival, +1)` and `(departure, −1)`, the answer is the maximum running sum. The statement guarantees distinct times; if it did not, the tie rule above would decide the answer.

3.2 Example: CSES 1740 — Intersection Points

Given $n \le 10^5$ horizontal and vertical segments with coordinates in $[-10^6, 10^6]$ (parallel segments never touch), count the intersection points. Sweep x from left to right. A horizontal segment at height y from $x1$ to $x2$ is "active" for $x \in [x1, x2]$: add $+1$ at y at $x1$, -1 at $x2$. A vertical segment at x from $y1$ to $y2$ queries the number of active heights in $[y1, y2]$ with a Fenwick tree. Because touching counts as intersecting, events at equal x are ordered: add, then query, then remove. Coordinates are small enough to offset by $10^6 + 1$ instead of compressing. O(n log C).

3.3 Example: CSES 1741 — Area of Rectangles

Compute the area of the union of $n \le 10^5$ axis-parallel rectangles. Sweep x. Between consecutive event coordinates, the union's cross-section is a fixed set of y-intervals, so

```
area += (covered y-length) × (x_next − x_current).
```

Each rectangle `[x1, x2] × [y1, y2]` adds `+1` to the y-range at `x1` and `−1` at `x2`. We need "range add ±1" and "total length of positions with count > 0".

The **cover-count segment tree** does this without lazy propagation. Each node stores `cnt` (how many rectangles cover the node's whole interval, counted only at the nodes where the update stopped) and `len` (covered length within the node). After an update, recompute `len` with `pull`: if `cnt > 0` the node is fully covered; otherwise it is the sum of its children (or 0 for a leaf). No push-down is needed because every `−1` exactly cancels an earlier `+1` on the same canonical set of nodes. The root's `len` is the covered length.

The leaves are elementary y-intervals of the compressed coordinates. The area can reach $(2 \cdot 10^6)^2 = 4 \cdot 10^{12}$, so use `long long`. The same tree gives the **perimeter** of the union with a little more bookkeeping (Codeforces 610D *Vika and Segments* is the "count unit cells covered by segments" variant).

4. Offline queries

If all queries are known in advance and there are no interleaved updates that depend on answers, you may reorder them. Sort the queries by one coordinate and sweep, so that each query sees exactly the right prefix of the data.

4.1 Example: CSES 1734 — Distinct Values Queries

For each query `[a, b]`, count distinct values in `x_a … x_b`. Sort queries by `b`. Sweep `r = 1 … n`, maintaining a Fenwick tree with a 1 at the **last occurrence so far** of each value (move the 1 when a value reappears). When `r = b`, the distinct values in `[a, b]` are exactly the values whose last occurrence up to `b` lies in `[a, b]`, so the answer is a range sum. O((n + q) log n).

Codeforces 1000F *One Occurrence* (find any value that occurs exactly once in a range) uses the same sweep with a segment tree storing, for each position whose value's last occurrence it is, the position of the previous occurrence; a range-min query answers it.

4.2 Nested ranges

CSES 2168/2169 Nested Ranges: for each range, does it contain another range, and is it contained in another (and how many)? Sort by left endpoint ascending and, for equal left endpoints, by right endpoint **descending**. Then any range containing range `i` appears before `i`, so "is `i` contained?" asks whether some earlier range has right end ≥ `r_i`, counted with a

Fenwick tree over compressed right endpoints. Symmetrically, sweep from the back for "contains". Identical ranges contain each other and need care with ties; work through the sample with duplicate ranges before coding.

5. Segment-tree patterns

5.1 Descent (walking down the tree)

CSES 1143 Hotel Queries: hotels have free rooms `h_i`; each group needs `r` rooms in a single hotel, the first (leftmost) hotel with enough rooms. Keep a max segment tree. From the root, go left if the left child's maximum is at least `r`, else right. One O(log n) walk finds the leftmost index, compared with O(log² n) for binary search over prefix maxima. The same technique finds the k-th one in a 0/1 array, the first prefix sum exceeding a value, and so on.

5.2 Mergeable nodes

Store a small struct in each node and write one `merge` function. Maximum subarray sum with point updates (CSES 1190) stores `(sum, best prefix, best suffix, best)`:

```
Node merge(Node a, Node b) {
    return {a.sum + b.sum,
            max(a.pre, a.sum + b.pre),
            max(b.suf, b.sum + a.suf),
            max({a.best, b.best, a.suf + b.pre})};
}
```

Other examples: longest run of equal values, bracket sequence balance `(unmatched open, unmatched close)`, and polynomial hashes of a range for string comparisons with updates (Lecture 9).

5.3 Trees over values

Index the tree by compressed *value* instead of position. Sweeping over positions and inserting values gives dominance counting in 2D: "how many earlier points have smaller value", inversions, or *Codeforces 1311F Moving Points* (sum of pairwise minimum distances when points move with constant speeds: sort by position and query a Fenwick tree over compressed speeds for count and sum of earlier positions with speed ≤ current).

5.4 Choosing the structure

Need	Structure
Prefix sums, point updates	Fenwick tree

Need	Structure
Static range min/max	sparse table
Range add + range sum/min	lazy segment tree
Union length/area	cover-count tree
First index with a property	segment tree descent
Ordered set operations	`std::set` / policy-based tree

6. Reference implementations

6.1 CSES 1741 — Area of Rectangles

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int M;
vector<int> ys, cnt;
vector<ll> len;

void pull(int node, int l, int r) {
    if (cnt[node] > 0) len[node] = ys[r + 1] - ys[l];
    else if (l == r) len[node] = 0;
    else len[node] = len[2 * node] + len[2 * node + 1];
}

void update(int node, int l, int r, int ql, int qr, int v) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { cnt[node] += v; pull(node, l, r); return; }
    int mid = (l + r) / 2;
    update(2 * node, l, mid, ql, qr, v);
    update(2 * node + 1, mid + 1, r, ql, qr, v);
    pull(node, l, r);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<int, 4>> ev;                 // (x, +1/-1, y1, y2)
    for (int i = 0; i < n; i++) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        ev.push_back({x1, +1, y1, y2});
        ev.push_back({x2, -1, y1, y2});
        ys.push_back(y1);
        ys.push_back(y2);
    }
```

```
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    M = (int)ys.size() - 1;                  // leaf i = [ys[i], ys[i+1]]
    cnt.assign(4 * max(M, 1), 0);
    len.assign(4 * max(M, 1), 0);
    sort(ev.begin(), ev.end());
    ll area = 0;
    for (size_t i = 0; i < ev.size(); i++) {
        if (i > 0) area += len[1] * (ev[i][0] - ev[i - 1][0]);
        int a = lower_bound(ys.begin(), ys.end(), ev[i][2]) - ys.begin();
        int b = lower_bound(ys.begin(), ys.end(), ev[i][3]) - ys.begin();
        if (a < b) update(1, 0, M - 1, a, b - 1, ev[i][1]);
    }
    cout << area << '\n';
}
```

Verified against cell-by-cell brute force on random small inputs; the CSES sample gives 24.

6.2 CSES 1734 — Distinct Values Queries (offline + Fenwick)

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n; vector<int> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
    int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> x(n + 1);
    for (int i = 1; i <= n; i++) cin >> x[i];
    vector<array<int, 3>> qs(q);                      // (b, a, id)
    for (int i = 0; i < q; i++) { cin >> qs[i][1] >> qs[i][0]; qs[i][2] = i; }
    sort(qs.begin(), qs.end());
    Fenwick fw(n);
    map<int, int> lastPos;
    vector<int> ans(q);
    int r = 0;
    for (auto [b, a, id] : qs) {
        while (r < b) {
            r++;
            auto it = lastPos.find(x[r]);
            if (it != lastPos.end()) fw.add(it->second, -1);
            lastPos[x[r]] = r;
            fw.add(r, +1);
        }
        ans[id] = fw.sum(b) - fw.sum(a - 1);
```

```
    }
    for (int v : ans) cout << v << '\n';
}
```

For the tightest time limits, replace the `map` by compressing values into a `vector<int>`.

6.3 CSES 1740 — Intersection Points

```
#include <bits/stdc++.h>
using namespace std;

const int OFF = 1'000'001;                          // coordinates in [-1e6, 1e6]
struct Fenwick {
    int n; vector<int> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
    int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<int, 4>> ev;                       // (x, type, y1, y2)
    for (int i = 0; i < n; i++) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        if (y1 == y2) {                             // horizontal
            if (x1 > x2) swap(x1, x2);
            ev.push_back({x1, 0, y1, 0});           // 0 = add
            ev.push_back({x2, 2, y1, 0});           // 2 = remove
        } else {                                    // vertical: 1 = query
            if (y1 > y2) swap(y1, y2);
            ev.push_back({x1, 1, y1, y2});
        }
    }
    sort(ev.begin(), ev.end());
    Fenwick fw(2 * OFF + 1);
    long long total = 0;
    for (auto [x, type, y1, y2] : ev) {
        if (type == 0) fw.add(y1 + OFF, +1);
        else if (type == 2) fw.add(y1 + OFF, -1);
        else total += fw.sum(y2 + OFF) - fw.sum(y1 + OFF - 1);
    }
    cout << total << '\n';
}
```

6.4 CSES 1143 — Hotel Queries (segment tree descent)

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> mx;

void build(const vector<int>& h, int node, int l, int r) {
    if (l == r) { mx[node] = h[l]; return; }
    int mid = (l + r) / 2;
    build(h, 2 * node, l, mid);
    build(h, 2 * node + 1, mid + 1, r);
    mx[node] = max(mx[2 * node], mx[2 * node + 1]);
}

int takeFirst(int node, int l, int r, int need) {   // leftmost index with value >= need
    if (mx[node] < need) return -1;
    if (l == r) { mx[node] -= need; return l; }
    int mid = (l + r) / 2;
    int res = (mx[2 * node] >= need) ? takeFirst(2 * node, l, mid, need)
                                     : takeFirst(2 * node + 1, mid + 1, r, need);
    mx[node] = max(mx[2 * node], mx[2 * node + 1]);
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> n >> m;
    vector<int> h(n);
    for (auto& x : h) cin >> x;
    mx.assign(4 * n, 0);
    build(h, 1, 0, n - 1);
    for (int i = 0; i < m; i++) {
        int r;
        cin >> r;
        int idx = takeFirst(1, 0, n - 1, r);
        cout << idx + 1 << (i + 1 == m ? '\n' : ' ');   // 0 means no hotel
    }
}
```

All four programs were stress-tested against brute-force solutions on random small inputs.

7. Common mistakes

- **Wrong event order** at equal coordinates (touching segments, closed vs. half-open intervals).
- **Compressing points when lengths matter** instead of elementary intervals.

- **Off-by-one** in elementary intervals: segment `[a, b]` covers leaves `id(a) … id(b) − 1`.
- **Overflow** in areas and in counts of pairs.
- **Forgetting query coordinates** in the compression list.
- **0-based Fenwick indices** (index 0 loops forever in `add`).
- **Lazy propagation where none is needed** (cover counts), or a missing push-down where it is needed (range assignment).
- **Recursion-heavy trees with 4n memory** for n = 10^6 in tight memory limits; consider an iterative bottom-up tree.

8. Practice problems

- CSES 1619 — Restaurant Customers: https://cses.fi/problemset/task/1619
- CSES 1740 — Intersection Points: https://cses.fi/problemset/task/1740
- CSES 1741 — Area of Rectangles: https://cses.fi/problemset/task/1741
- CSES 1734 — Distinct Values Queries: https://cses.fi/problemset/task/1734
- CSES 1143 — Hotel Queries: https://cses.fi/problemset/task/1143
- CSES 1144 — Salary Queries: https://cses.fi/problemset/task/1144
- CSES 2168 / 2169 — Nested Ranges Check / Count: https://cses.fi/problemset/task/2168
- CSES 1190 — Subarray Sum Queries: https://cses.fi/problemset/task/1190
- Codeforces 610D — Vika and Segments: https://codeforces.com/problemset/problem/610/D
- Codeforces 1000F — One Occurrence: https://codeforces.com/problemset/problem/1000/F
- Codeforces 652D — Nested Segments: https://codeforces.com/problemset/problem/652/D
- Codeforces 1311F — Moving Points: https://codeforces.com/problemset/problem/1311/F
- AtCoder ABC 221 D — Online games: https://atcoder.jp/contests/abc221/tasks/abc221_d

9. Further reading

- cp-algorithms: "Segment Tree" (https://cp-algorithms.com/data_structures/segment_tree.html), especially "searching for the first element greater than a given amount" and "saving the entire subarrays in each vertex".
- de Berg, Cheong, van Kreveld, Overmars — *Computational Geometry: Algorithms and Applications*, Chapter 2 (plane sweep).
- Year Curriculum Weeks 23–25; National track Lecture 2 (persistent and merge-sort trees) builds on these patterns.